

《优化理论与算法》第 6 次作业 solution

1 习题

1.1 给定一个有向网络 $G = (V, E)$ ，其中顶点集合 V 包含 6 个节点，分别记为

$$\{s, 1, 2, 3, 4, t\},$$

其中 s 为源点， t 为汇点。下表给出了网络中各条有向边及其容量：

边	容量
$(s, 1)$	15
$(s, 2)$	5
$(1, 2)$	5
$(1, 3)$	10
$(2, 4)$	8
$(3, 4)$	7
$(3, t)$	6
$(4, t)$	12

请根据以上数据完成下列要求：

1. **模型建立：** 建立最大流问题模型，建立最小割问题模型
2. **求解：** 使用任何合适的求解器，求解上述最大流模型，请给出最优解中各条边的流量分配结果；求解上述最小割模型，请给出最优分割。最后，验证最大流最小割定理成立。
3. **结果分析与讨论：**
 - (a) 若将边 $(1, 3)$ 的容量由 10 调整为 5，最大流量是否会发生变化？若会，新的最优流量值是多少？
 - (b) 若在节点 2 与节点 3 之间增设一条新边 $(2, 3)$ ，容量为 6，则新的最优流量是多少？

1.1 solution

1.1.1

最大流问题模型

变量定义: - $f_{u,v}$: 边 (u, v) 上的流量。目标函数:

$$\text{Maximize } f = \sum_{v \in V} f_{s,v} - \sum_{v \in V} f_{v,s} \quad (\text{即从源点 } s \text{ 流出的净流量})$$

约束条件:

容量约束:

$$0 \leq f_{u,v} \leq c(u, v) \quad \forall (u, v) \in E$$

流量守恒:

$$\sum_{v \in V} f_{v,u} = \sum_{v \in V} f_{u,v} \quad \forall u \in V \setminus \{s, t\}$$

最小割问题模型

决策变量

$$z_i = \begin{cases} 1, & \text{if } i \in V_s, \\ 0, & \text{otherwise.} \end{cases}$$

$$y_{ij} = \begin{cases} 1, & \text{if } i \in V_s, j \in V_t, \\ 0, & \text{otherwise.} \end{cases}$$

目标函数:

$$\text{minimize } \sum_{(i,j) \in E} u_{ij} y_{ij}$$

约束条件:

1. $y_{ij} = \max\{z_i - z_j, 0\}, \quad \forall (i, j) \in E$
2. $z_s = 1, \quad z_t = 0$
3. $z_i \in \{0, 1\}, \quad \forall i \in V$

1.1.2

最大流与最小割都为 18

最大流

```

In [5]: import coptpy
        from coptpy import *

        env = coptpy.Envr()

        # 创建模型
        model = env.createModel("Graph")

        # 定义节点和边
        nodes = ["s", "1", "2", "3", "4", "t"]
        edges = {
            ("s", "1"): 15,
            ("s", "2"): 5,
            ("1", "2"): 5,
            ("1", "3"): 10,
            ("2", "4"): 8,
            ("3", "4"): 7,
            ("3", "t"): 6,
            ("4", "t"): 12,
        }

        # 添加变量：每条边的流量
        f = {}
        for (u, v) in edges:
            f[u, v] = model.addVar(lb=0, ub=edges[u, v], name=f"f_{u}_{v}")

        # 目标函数：最大化从源点流出的流量
        model.setObjective(quicksum(f["s", v] for v in nodes if ("s", v) in edges), COPT

        # 流量守恒约束（中间节点）
        for u in nodes:
            if u not in ["s", "t"]:
                inflow = quicksum(f[v, u] for (v, w) in edges if w == u)
                outflow = quicksum(f[u, w] for (v, w) in edges if v == u)
                model.addConstr(inflow == outflow, name=f"flow_conservation_{u}")

        # 求解
        model.setParam('logging', 0)

        model.solve()

        # 输出结果
        print("最大流值:", model.objVal)
        for (u, v) in edges:
            print(f"边 {u}->{v} 的流量: {f[u, v].X}")

```

最大流值: 18.0
边 s->1 的流量: 15.0
边 s->2 的流量: 3.0
边 1->2 的流量: 5.0
边 1->3 的流量: 10.0
边 2->4 的流量: 8.0
边 3->4 的流量: 4.0
边 3->t 的流量: 6.0
边 4->t 的流量: 12.0
最大流值: 18.0
边 s->1 的流量: 15.0
边 s->2 的流量: 3.0
边 1->2 的流量: 5.0
边 1->3 的流量: 10.0
边 2->4 的流量: 8.0
边 3->4 的流量: 4.0
边 3->t 的流量: 6.0
边 4->t 的流量: 12.0

最小割

```
In [6]: import coptpy
        from coptpy import *

        env = coptpy.Envr()

        # 创建模型
        model = env.createModel("Graph")

        # 定义节点和边
        nodes = ["s", "1", "2", "3", "4", "t"]
        edges = {
            ("s", "1"): 15,
            ("s", "2"): 5,
            ("1", "2"): 5,
            ("1", "3"): 10,
            ("2", "4"): 8,
            ("3", "4"): 7,
            ("3", "t"): 6,
            ("4", "t"): 12,
        }

        # 添加变量: y_u表示节点u是否属于S集合 (二进制变量)
        y = {}
        for u in nodes:
            y[u] = model.addVar(vtype=COPT.BINARY, name=f"y_{u}")

        # 添加变量w_uv表示边(u, v)是否在割中 (即u在S, v在T)
        w = {}
        for (u, v) in edges:
            w[u, v] = model.addVar(vtype=COPT.BINARY, name=f"w_{u}_{v}")

        # 目标函数: 最小化割的容量
```

```

model.setObjective(quicksum(edges[u, v] * w[u, v] for (u, v) in edges), COPT.MIN

# 添加约束:  $y_s = 1, y_t = 0$ 
model.addConstr(y["s"] == 1, name="y_s")
model.addConstr(y["t"] == 0, name="y_t")

# 添加约束:  $w_{uv} = y_u * (1 - y_v)$ 
# 添加约束:  $w_{uv} = y_u * (1 - y_v)$ 
for (u, v) in edges:
    model.addConstr(w[u, v] <= y[u], name=f"w_constr1_{u}_{v}")
    model.addConstr(w[u, v] <= 1 - y[v], name=f"w_constr2_{u}_{v}")
    model.addConstr(w[u, v] >= y[u] + (1 - y[v]) - 1, name=f"w_constr3_{u}_{v}")

# 求解
model.setParam('logging', 0)

model.solve()

# 输出结果
print("最小割容量:", model.objVal)
print("割集 S 包含的节点: ", [u for u in nodes if y[u].X > 0.5])

```

Cardinal Optimizer v7.2.1. Build date Oct 17 2024
Copyright Cardinal Operations 2024. All Rights Reserved

最小割容量: 18.0
割集 S 包含的节点: ['s', '1', '2', '3', '4']
最小割容量: 18.0
割集 S 包含的节点: ['s', '1', '2', '3', '4']

1.1.3

第一问答案: 13

调整边

```

In [7]: import coptpy
from coptpy import *

env = coptpy.Envr()

# 创建模型
model = env.createModel("Graph")

# 定义节点和边
nodes = ["s", "1", "2", "3", "4", "t"]
edges = {
    ("s", "1"): 15,
    ("s", "2"): 5,
    ("1", "2"): 5,
    ("1", "3"): 5, # 修改数值为5
    ("2", "4"): 8,
    ("3", "4"): 7,
    ("3", "t"): 6,
    ("4", "t"): 12,
}

# 添加变量: 每条边的流量
f = {}
for (u, v) in edges:
    f[u, v] = model.addVar(lb=0, ub=edges[u, v], name=f"f_{u}_{v}")

```

```

# 目标函数：最大化从源点流出的流量
model.setObjective(quicksum(f["s", v] for v in nodes if ("s", v) in edges), COPT

# 流量守恒约束（中间节点）
for u in nodes:
    if u not in ["s", "t"]:
        inflow = quicksum(f[v, u] for (v, w) in edges if w == u)
        outflow = quicksum(f[u, w] for (v, w) in edges if v == u)
        model.addConstr(inflow == outflow, name=f"flow_conservation_{u}")

# 求解
model.setParam('logging', 0)

model.solve()

# 输出结果
print("最大流值:", model.objVal)
for (u, v) in edges:
    print(f"边 {u}->{v} 的流量: {f[u, v].X}")

```

Cardinal Optimizer v7.2.1. Build date Oct 17 2024
Copyright Cardinal Operations 2024. All Rights Reserved

```

最大流值: 13.0
边 s->1 的流量: 10.0
边 s->2 的流量: 3.0
边 1->2 的流量: 5.0
边 1->3 的流量: 5.0
边 2->4 的流量: 8.0
边 3->4 的流量: 0.0
边 3->t 的流量: 5.0
边 4->t 的流量: 8.0
最大流值: 13.0
边 s->1 的流量: 10.0
边 s->2 的流量: 3.0
边 1->2 的流量: 5.0
边 1->3 的流量: 5.0
边 2->4 的流量: 8.0
边 3->4 的流量: 0.0
边 3->t 的流量: 5.0
边 4->t 的流量: 8.0

```

第二问答案：18

添加边

```

In [8]: import coptpy
        from coptpy import *

        env = coptpy.Envr()

        # 创建模型
        model = env.createModel("Graph")

        # 定义节点和边
        nodes = ["s", "1", "2", "3", "4", "t"]
        edges = {
            ("s", "1"): 15,

```

```

    ("s", "2"): 5,
    ("1", "2"): 5,
    ("1", "3"): 10,
    ("2", "4"): 8,
    ("3", "4"): 7,
    ("3", "t"): 6,
    ("4", "t"): 12,
}

edges[("2", "3")] = 6 # 新增边

# 添加变量: 每条边的流量
f = {}
for (u, v) in edges:
    f[u, v] = model.addVar(lb=0, ub=edges[u, v], name=f"f_{u}_{v}")

# 目标函数: 最大化从源点流出的流量
model.setObjective(quicksum(f["s", v] for v in nodes if ("s", v) in edges), COPT

# 流量守恒约束 (中间节点)
for u in nodes:
    if u not in ["s", "t"]:
        inflow = quicksum(f[v, u] for (v, w) in edges if w == u)
        outflow = quicksum(f[u, w] for (v, w) in edges if v == u)
        model.addConstr(inflow == outflow, name=f"flow_conservation_{u}")

# 求解
model.setParam('logging', 0)
model.solve()

# 输出结果
print("最大流值:", model.objVal)
for (u, v) in edges:
    print(f"边 {u}->{v} 的流量: {f[u, v].X}")

```

```

最大流值: 18.0
边 s->1 的流量: 15.0
边 s->2 的流量: 3.0
边 1->2 的流量: 5.0
边 1->3 的流量: 10.0
边 2->4 的流量: 8.0
边 3->4 的流量: 4.0
边 3->t 的流量: 6.0
边 4->t 的流量: 12.0
边 2->3 的流量: 0.0
最大流值: 18.0
边 s->1 的流量: 15.0
边 s->2 的流量: 3.0
边 1->2 的流量: 5.0
边 1->3 的流量: 10.0
边 2->4 的流量: 8.0
边 3->4 的流量: 4.0
边 3->t 的流量: 6.0
边 4->t 的流量: 12.0
边 2->3 的流量: 0.0
    
```

1.2 试证明：引入容量下界的最小成本流问题，也符合整数定理，即：如果如下线性规划中，

$$\begin{aligned}
 & \text{minimize} && \sum_{(i,j) \in E} c_{ij} x_{ij} \\
 & \text{subject to} && \sum_{(k,i) \in E} x_{ki} - \sum_{(i,j) \in E} x_{ij} = d_i, && \forall i \in V \\
 & && l_{ij} \leq x_{ij} \leq u_{ij}, && \forall (i,j) \in E
 \end{aligned}$$

参数 d_i, l_{ij}, u_{ij} 均为整数，那么其所有基本可行解均为整数解。

1.2 solution

原问题的约束为：

- 流量守恒方程： $A\mathbf{x} = \mathbf{d}$ ，其中 A 是点边关联矩阵。
- 容量约束： $l_{ij} \leq x_{ij} \leq u_{ij}$ 。

令 $\mathbf{x} = \mathbf{l} + \mathbf{y}$ ，其中 $\mathbf{y} \geq 0$ 。此时：

- 下界约束 $\mathbf{x} \geq \mathbf{l}$ 自动满足。
- 上界约束变为 $\mathbf{l} + \mathbf{y} \leq \mathbf{u}$ ，即 $\mathbf{y} \leq \mathbf{u} - \mathbf{l}$ 。

将原问题转化为：

$$\begin{aligned} & \text{minimize} && \mathbf{c}^T(\mathbf{l} + \mathbf{y}) \\ & \text{subject to} && A\mathbf{y} = \mathbf{d} - A\mathbf{l}, \\ & && 0 \leq \mathbf{y} \leq \mathbf{u} - \mathbf{l}. \end{aligned}$$

这是一个标准的最小成本流问题，变量为 $\mathbf{y}$ ，容量上界为 $\mathbf{u} - \mathbf{l}$ ，右侧向量为 $\mathbf{d} - A\mathbf{l}$ 。
因此：

- 若 $\mathbf{d}, \mathbf{l}, \mathbf{u}$ 均为整数，则 $\mathbf{d} - A\mathbf{l}$ 和 $\mathbf{u} - \mathbf{l}$ 也为整数（因 A 的元素为 -1, 0, +1）。
- 标准最小成本流问题的约束矩阵 A 是全单模矩阵，且右侧向量为整数，因此所有基本可行解 $\mathbf{y}$ 均为整数。
- 由 $\mathbf{x} = \mathbf{l} + \mathbf{y}$ ，且 $\mathbf{l}$ 为整数，故 $\mathbf{x}$ 也为整数。